

# Preserves: an Expressive Data Language

Tony Garnock-Jones

## Preserves: an Expressive Data Language

Tony Garnock-Jones [tonyg@leastfixedpoint.com](mailto:tonyg@leastfixedpoint.com)

June 2025. Version 0.996.3.

*Preserves* is a data model, with associated serialization formats.

It supports *records* with user-defined *labels*, embedded *references*, and the usual suite of atomic and compound data types, including *binary* data as a distinct type from text strings. Its *annotations* allow separation of data from metadata such as comments, trace information, and provenance information.

Preserves departs from many other data languages in defining how to *compare* two values. Comparison is based on the data model, not on syntax or on data structures of any particular implementation language.

This document defines the core semantics and data model of Preserves and presents a handful of examples. Two other core documents define

- a human-readable text syntax, and
- a machine-oriented binary syntax

for the Preserves data model.

## Values

Preserves *values* are given meaning independent of their syntax. We will write “**Value**” when we mean the set of all Preserves values or an element of that set.

Values fall into two broad categories: *atomic* and *compound* data. Every **Value** is finite and non-cyclic. Embedded values, called **Embeddeds**, are a third, special-case category.

```
Value = Atom
      | Compound
      | Embedded

Atom = Boolean
     | Double
     | SignedInteger
     | String
     | ByteString
     | Symbol

Compound = Record
         | Sequence
         | Set
         | Dictionary
```

**Total order.** As we go, we will incrementally specify a total order over **Values**. Two values of the same kind are compared using kind-specific rules. The ordering among values of different kinds is essentially arbitrary,

but having a total order is convenient for many tasks, so we define it as follows:

|             |                                                                    |
|-------------|--------------------------------------------------------------------|
| (Values)    | Atom < Compound < Embedded                                         |
| (Compounds) | Record < Sequence < Set < Dictionary                               |
| (Atoms)     | Boolean < Double < SignedInteger<br>< String < ByteString < Symbol |

**Equivalence.** Two `Values` are equal if neither is less than the other according to the total order.

### Signed integers.

A `SignedInteger` is an arbitrarily-large signed integer. `SignedIntegers` are compared as mathematical integers.

### Unicode strings.

A `String` is a sequence of Unicode scalar values.<sup>1</sup> `Strings` are compared lexicographically, scalar value by scalar value.<sup>2</sup>

### Binary data.

A `ByteString` is a sequence of octets. `ByteStrings` are compared lexicographically.

### Symbols.

Programming languages like Lisp and Prolog frequently use string-like values called *symbols*.<sup>3 4</sup> Here, a `Symbol` is, like a `String`, a sequence of Unicode scalar values representing an identifier of some kind. `Symbols` are also compared lexicographically by scalar value.

### Booleans.

There are two `Booleans`, “false” and “true”. The “false” value is less-than the “true” value.

### IEEE floating-point values.

`Doubles` are double-precision IEEE 754 floating-point values.<sup>5</sup> `Doubles` and `SignedIntegers` are disjoint; by the rules above, every `Double` is less than every `SignedInteger`. Two `Doubles` are to be ordered by the `totalOrder` predicate defined in section 5.10 of IEEE Std 754-2008.

### Records.

A `Record` is a *labelled* tuple of `Values`, the record’s *fields*. A label can be any `Value`, but is usually a `Symbol`.<sup>6</sup>  
<sup>7</sup> `Records` are ordered first by label, then lexicographically<sup>8</sup> by field sequence.

### Sequences.

A `Sequence` is a sequence of `Values`. `Sequences` are compared lexicographically.<sup>8</sup>

### Sets.

A `Set` is an unordered finite set of `Values`. It contains no duplicate values, following the equivalence relation induced by the total order on `Values`. Two `Sets` are compared by sorting their elements ascending using the total order and comparing the resulting `Sequences`.<sup>8</sup>

## Dictionaries.

A **Dictionary** is an unordered finite collection of pairs of **Values**. Each pair comprises a *key* and a *value*. Keys in a **Dictionary** are pairwise distinct. Instances of **Dictionary** are compared by lexicographic<sup>8</sup> comparison of the sequences resulting from ordering each **Dictionary**'s pairs in ascending order by key.

## Embeddeds.

An **Embedded** allows inclusion of *domain-specific*, potentially *stateful* or *located* data into a **Value**.<sup>9</sup> **Embeddeds** may be used to denote stateful objects, network services, object capabilities, file descriptors, Unix processes, or other possibly-stateful things. Because each **Embedded** is a domain-specific datum, comparison of two **Embeddeds** is done according to domain-specific rules.

*Motivating Examples.* In a Java or Python implementation, an **Embedded** may denote a reference to a Java or Python object; comparison would be done via the language's own rules for equivalence and ordering. In a Unix application, an **Embedded** may denote an open file descriptor or a process ID. In an HTTP-based application, each **Embedded** might be a URL, compared according to RFC 6943. When a **Value** is serialized for storage or transfer, **Embeddeds** will usually be represented as ordinary **Values**, in which case the ordinary rules for comparing **Values** will apply.

## Appendix. Examples

The definitions above are independent of any particular concrete syntax. The examples of **Values** that follow are written using the Preserves text syntax, and the example encoded byte sequences use the Preserves binary encoding.

## Ordering.

The total ordering specified above means that the following statements are true:

- "bzz" < "c" < "caa" < #: "a"
- #t < 3.0 < 3 < "3" < '3' < [] < #: #t
- [#f] < [foo], because **Boolean** appears before **Symbol** in the kind ordering
- [x] < [x y], because there is no element remaining to compare against y
- [a b] < [x], because a is smaller than x
- [x y] < [x z], because y is ordered before z

## Simple examples.

| Value                                           | Encoded byte sequence                                                           |
|-------------------------------------------------|---------------------------------------------------------------------------------|
| <capture <discard>>                             | B4 B3 07 'c' 'a' 'p' 't' 'u' 'r' 'e' B4 B3 07 'd' 'i' 's' 'c' 'a' 'r' 'd' 84 84 |
| [1 2 3 4]                                       | B5 B0 01 01 B0 01 02 B0 01 03 B0 01 04 84                                       |
| [-2 -1 0 1]                                     | B5 B0 01 FE B0 01 FF B0 00 B0 01 01 84                                          |
| "hello"                                         | B1 05 'h' 'e' 'l' 'l' 'o'                                                       |
| "z "                                            | B1 08 'z' E6 B0 B4 F0 9D 84 9E                                                  |
| "z \uD834\uDD1E"                                | B1 08 'z' E6 B0 B4 F0 9D 84 9E                                                  |
| ["a" b #"c" [] #{} #t #f]                       | B5 B1 01 'a' B3 01 'b' B2 01 'c' B5 84 B6 84 81 80 84                           |
| -257                                            | B0 02 FE FF                                                                     |
| -1                                              | B0 01 FF                                                                        |
| 0                                               | B0 00                                                                           |
| 1                                               | B0 01 01                                                                        |
| 255                                             | B0 02 00 FF                                                                     |
| 1.0                                             | 87 08 3F F0 00 00 00 00 00 00                                                   |
| -1.202e300                                      | 87 08 FE 3C B7 B7 59 BF 04 26                                                   |
| #xd"fff0000000000000", negative Double infinity | 87 08 FF F0 00 00 00 00 00 00                                                   |

The next example uses a non-Symbol label for a record.<sup>10</sup> The Record  
 <[titled person 2 thing 1] 101 "Blackwell" <date 1821 2 3> "Dr">  
 encodes to

```

B4                                     # Record
  B5                                   # Sequence
    B3 06 74 69 74 6C 65 64          # Symbol, "titled"
    B3 06 70 65 72 73 6F 6E          # Symbol, "person"
    B0 01 02                          # SignedInteger, "2"
    B3 05 74 68 69 6E 67              # Symbol, "thing"
    B0 01 01                          # SignedInteger, "1"
  84                                  # End (sequence)
  B0 01 65                            # SignedInteger, "101"
  B1 09 42 6C 61 63 6B 77 65 6C 6C    # String, "Blackwell"
  B4                                     # Record
    B3 04 64 61 74 65                # Symbol, "date"
    B0 02 07 1D                      # SignedInteger, "1821"
    B0 01 02                          # SignedInteger, "2"
    B0 01 03                          # SignedInteger, "3"
  84                                  # End (record)
  B1 02 44 72                        # String, "Dr"
84                                   # End (record)

```

## JSON examples.

Preserves text syntax is a superset of JSON,<sup>11</sup> so the examples from RFC 8259 read as valid Preserves.

The JSON literals `true`, `false` and `null` all read as Symbols, and JSON numbers read (unambiguously) either as SignedIntegers or as Doubles.<sup>12</sup>

The first RFC 8259 example:

```

{
  "Image": {
    "Width": 800,
    "Height": 600,
    "Title": "View from 15th Floor",
    "Thumbnail": {
      "Url": "http://www.example.com/image/481989943",
      "Height": 125,
      "Width": 100
    },
    "Animated" : false,
    "IDs": [116, 943, 234, 38793]
  }
}

```

when read using the Preserves text syntax encodes via the binary syntax as follows:

```

B7
  B1 05 "Image"
  B7
    B1 03 "IDs"      B5
                      B0 01 74
                      B0 02 03 AF
                      B0 02 00 EA

```

```

                                B0 03 00 97 89
                                84
B1 05 "Title"      B1 14 "View from 15th Floor"
B1 05 "Width"      B0 02 03 20
B1 06 "Height"     B0 02 02 58
B1 08 "Animated"   B3 05 "false"
B1 09 "Thumbnail"
    B7
      B1 03 "Url"      B1 26 "http://www.example.com/image/481989943"
      B1 05 "Width"    B0 01 64
      B1 06 "Height"   B0 01 7D
    84
84

```

If Preserves had been used directly, it would have been more idiomatic to use **Symbols** instead of **Strings** for the field labels.

The second RFC 8259 example:

```

[
  {
    "precision": "zip",
    "Latitude": 37.7668,
    "Longitude": -122.3959,
    "Address": "",
    "City": "SAN FRANCISCO",
    "State": "CA",
    "Zip": "94107",
    "Country": "US"
  },
  {
    "precision": "zip",
    "Latitude": 37.371991,
    "Longitude": -122.026020,
    "Address": "",
    "City": "SUNNYVALE",
    "State": "CA",
    "Zip": "94085",
    "Country": "US"
  }
]

```

encodes to binary as follows:

```

B5
  B7
    B1 03 "Zip"          B1 05 "94107"
    B1 04 "City"         B1 0D "SAN FRANCISCO"
    B1 05 "State"        B1 02 "CA"
    B1 07 "Address"      B1 00
    B1 07 "Country"      B1 02 "US"
    B1 08 "Latitude"     87 08 40 42 E2 26 80 9D 49 52
    B1 09 "Longitude"    87 08 C0 5E 99 56 6C F4 1F 21
    B1 09 "precision"    B1 03 "zip"
  84

```

```

B7
B1 03 "Zip"          B1 05 "94085"
B1 04 "City"         B1 09 "SUNNYVALE"
B1 05 "State"        B1 02 "CA"
B1 07 "Address"      B1 00
B1 07 "Country"      B1 02 "US"
B1 08 "Latitude"     87 08 40 42 AF 9D 66 AD B4 03
B1 09 "Longitude"    87 08 C0 5E 81 AA 4F CA 42 AF
B1 09 "precision"    B1 03 "zip"
84
84

```

## Appendix. Merging Values

The *merge* of two **Values** is a combination of the two values that includes all information from each that is missing from the other. If the values are incompatible, they have no merge.

- the merge of two **Atoms** has no value if they are not equal; otherwise, it has value equal to (an arbitrary) one of the atoms.
- the merge of two **Embeddeds** depends on the interpretation of the embedded values, and so is implementation-defined.
- the merge of two **Compounds** is:
  - if both are **Sequences**, let **n** be the minimum of the lengths of the two sequences. If every merge of corresponding positions up to **n** in the sequences is defined, the result is defined, with elements merged up to position **n** and simply copied from the longer sequence from position **n** onward; otherwise, it is undefined.
  - if both are **Records**, the **Record** with the merge of the two input records' labels as its label and the merge of the inputs' field sequences as its fields;
  - if both are **Dictionarys**, if every merge of values associated with keys common to both inputs is defined, the result is defined, with merged values at common keys and simply copied from either side for keys unique to that side; otherwise, it is undefined.
- Otherwise, the merge is undefined.

### Examples.

- `merge [1, [2], 3] [1, [2, 99], 3, 4, 5] = [1, [2, 99], 3, 4, 5]`
- `merge [1, 2, 3] [1, 5, 3] =`
- `merge #{a, b, c} #{a, b, c} =`
- `merge {a: 1, b: [2]} {b: [2, 99] c: 3} = {a: 1, b: [2, 99], c: 3}`
- `merge {a: 1, b: [2]} {a: 5, b: [2]} =`

## Notes

1. All Unicode scalar values are permitted, including NUL (scalar value zero). Because scalar values are defined as code points *excluding* surrogate code points (D800<sub>16</sub>–DFFF<sub>16</sub>), surrogates are *not* permitted in Preserves Unicode data.
2. Happily, the design of UTF-8 is such that this gives the same result as a lexicographic byte-by-byte comparison of the UTF-8 encoding of a string!
3. Even Java has quasi-symbols in the form of its “interned strings”. A Java Preserves implementation might intern Preserves **Symbols** while leaving Preserves **Strings** uninterned.

4. **Rationale.** Why include `Symbols` when we already have `Strings`? Applications should use `Symbols` (instead of `Strings`) when the specific text is mostly *static*, used as part of the shape of a data structure, rather than as part of the value a data structure denotes. For example, when using a `Dictionary` as an object (as opposed to as a mapping), the keys should usually be `Symbols` because the set of fields in the modelled object is fixed and statically known, and the field names belong properly to the *type* and not the specific *value*. When using a `Dictionary` as, say, an address book mapping names to addresses, the keys should be `Strings` because names are part of the modelled data set. Similarly, `Symbols` will frequently appear as `Record` labels because they often denote data *types*, which are static and fixed.
  5. Every value inhabiting a smaller IEEE 754 type (e.g. single- or half-precision) can be injected into and projected from double-precision losslessly and in an order-preserving way.
  6. The Racket programming language defines “prefab” structure types, which map well to our `Records`. Racket supports record extensibility by encoding record supertypes into record labels as specially-formatted lists.
  7. It is occasionally (but seldom) necessary to interpret such `Symbol` labels as IRIs. Where a label can be read as a relative IRI, it is notionally interpreted with respect to the IRI `urn:uuid:6bf094a6-20f1-4887-ada7-46834a9b5b34`; where a label can be read as an absolute IRI, it stands for that IRI; and otherwise, it cannot be read as an IRI at all, and so the label simply stands for itself—for its own `Value`.
  8. When comparing sequences of values for the total order, lexicographical ordering is used. Elements are drawn pairwise from the two sequences to be compared. If one is smaller than the other according to the total order, the sequence it was drawn from is the smaller of the sequences. If the end of one sequence is reached, while the other sequence has elements remaining, the shorter sequence is considered smaller. Otherwise, all the elements compared equal and neither was longer than the other, so they compare equal. For example,
    - `[#f]` is ordered before `[foo]` because `Boolean` appears before `Symbol` in the kind ordering;
    - `[x]` before `[x y]` because there is no element remaining to compare against `y`;
    - `[a b]` before `[x]` because `a` is smaller than `x`; and
    - `[x y]` before `[x z]` because `y` is ordered before `z` according to the ordering rules for `Symbol`.
- 2 3 4
9. **Rationale.** Why include `Embeddeds` as a special class, distinct from, say, a specially-labeled `Record`? First, a `Record` can only hold other `Values`: in order to embed values such as live pointers to Java objects, some means of “escaping” from the `Value` data type must be provided. Second, `Embeddeds` are meant to be able to denote stateful entities, for which comparison by address is appropriate; however, we do not wish to place restrictions on the *nature* of these entities: if we had used `Records` instead of distinct `Embeddeds`, users would have to invent an encoding of domain data into `Records` that reflected domain ordering into `Value` ordering. This is often difficult and may not always be possible. Finally, because `Embeddeds` are intended to be able to represent network and memory *locations*, they must be able to be rewritten at network and process boundaries. Having a distinct class allows generic `Embedded` rewriting without the quotation-related complications of encoding references as, say, `Records`.
  10. It happens to line up with Racket’s representation of a record label for an inheritance hierarchy where `titled` extends `person` extends `thing`:
 

```
(struct date (year month day) #:prefab)
(struct thing (id) #:prefab)
(struct person thing (name date-of-birth) #:prefab)
(struct titled person (title) #:prefab)
```

For more detail on Racket’s representations of record labels, see the Racket documentation for `make-prefab-struct`.
  11. There is one caveat to be aware of. Section 8.2 of RFC 8259 explicitly permits unpaired surrogate code

points in JSON texts without specifying an interpretation for them. Preserves mandates UTF-8 in its binary syntax, forbids unpaired surrogates in its text syntax, and disallows surrogate code points in **Strings** and **Symbols**, meaning that any valid JSON text including an unpaired surrogate will not be parseable using the Preserves text syntax rules.

12. The following schema definitions match exactly the JSON subset of a Preserves input:

```
version 1 .
JSON = @string string / @integer int / @double double / @boolean JSONBoolean / @null =null
      / @array [JSON ...] / @object { string: JSON ...:... } .
JSONBoolean = =true / =false .
```